

H0006 — Ver categorías (Profesor)

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

Épica: Consulta Profesor · Puntos: 1 · Prioridad KANO: Básico · Rol: Profesor

Rama: Dev · Fecha: 2026-08-20 · Migración requerida: ninguna

1. Qué pedía la historia y qué faltaba de verdad

Como **Profesor**, poder **ver las categorías** de un torneo para saber cómo está organizada la competencia. Es una historia de 1 punto y buena parte parecía ya resuelta: `GET /api/v1/torneos/{torneoId}/categorias` existía desde H0002 y ya declaraba `Roles("Coordinador", "Profesor", "Director")`.

Al recorrer el flujo completo de un Profesor aparecieron **tres huecos reales**:

Hueco	Detalle
No existía el usuario Profesor	El seeder solo creaba al Coordinador. La historia era literalmente <i>imposible de usar o de probar</i> : no había credenciales con las que entrar como Profesor.
Faltaba el dato principal	La card de categoría mostraba nombre, tipo, sexo y rangos, pero no cuántos competidores tiene cada una. Para quien consulta (y no gestiona), ese es el dato central: saber si la categoría de sus alumnos tiene gente. Ya estaba especificado como <code>totalCompetidores</code> en <code>frontend/CLAUDE.md</code> , pero nunca se había implementado.
La página solo le hablaba al Coordinador	Todo el texto explicativo vivía dentro de bloques <code>esCoordinador && ...</code> . Un Profesor veía una pantalla sin acciones y sin ninguna explicación de qué estaba mirando.

2. Decisiones de diseño

D1 — El seeder pasa a verificar por email, uno por uno

Este fue el hallazgo más importante y no era evidente. El seeder empezaba así:

if (await usuarios.ExisteAlgunoAsync(ct)) return;

Es decir: **si ya hay algún usuario, no siembres nada**. Agregar el Profesor al arreglo de semillas *no habría alcanzado*: como la base de desarrollo ya tenía al Coordinador, la comprobación global cortaba de entrada y el Profesor nunca se habría creado. La única forma de obtenerlo habría sido borrar y recrear la base, perdiendo el torneo de prueba con el que se verificó H0005.

Solución: el seeder recorre una lista de semillas y para cada una consulta `GetByEmailAsync`. Si el usuario ya existe lo saltea; si no, lo crea. Pasa de ser *todo o nada, una sola vez* a ser **idempotente e incremental**: cada vez que se agregue un rol nuevo (el Director de H0008, por ejemplo), aparece solo al reiniciar la API, sin tocar la base.

Como consecuencia, `ExisteAlgunoAsync` quedó sin ningún uso y se eliminó de `IUsuarioRepository` y de su implementación, para no dejar código muerto.

D2 — `TotalCompetidores` es nullable, y eso significa algo

Se agregó `int? TotalCompetidores` a `CategoriaResponse`. Es **nullable a propósito**: solo el listado de categorías lo calcula. En las respuestas de crear y de editar una categoría llega en `null`.

La alternativa era un `int` con 0 por defecto, pero eso **miente**: al editar una categoría con 8 competidores, la respuesta diría 0 competidores. Con nullable, los dos casos quedan separados de forma explícita:

- `null` → no se calculó en esta respuesta
- `0` → esta categoría no tiene competidores

El frontend respeta esa distinción: la fila Competidores solo se dibuja si el valor no es `null`, y cuando vale 0 muestra *Ninguno todavía* en lugar de un cero suelto.

D3 — Un solo `GROUP BY`, no una consulta por categoría

El conteo se resuelve con un método nuevo del repositorio,

`ICompetidorRepository.GetConteoPorCategoriaAsync(torneoId, ct)`, que devuelve un `Dictionary<Guid, int>` y se traduce a un **único GROUP BY en SQL**.

Se descartaron dos alternativas:

- **Contar por categoría dentro del bucle** → sería un N+1: con 20 categorías, 21 consultas.
- **Traer todos los competidores y agrupar en memoria** → simple, pero mueve cientos de filas completas desde la base para terminar usando solo su cantidad.

Hay un test dedicado (`Handle_ConteoSeConsultaUnaSolaVezParaTodasLasCategorias`) que verifica que el repositorio se invoca **exactamente una vez**, para que un refactor futuro no reintroduzca el N+1 sin que nadie se entere.

D4 — El estado de las llaves se dice con palabras

Antes, que una categoría tuviera llaves generadas se comunicaba **solo por la presencia del enlace Ver llaves**. Para un Coordinador eso alcanza porque conoce el flujo; para un Profesor es ambiguo: la ausencia del enlace se puede leer como *todavía no hay llaves* o como *no tengo permiso para verlas*, que son cosas muy distintas.

La card ahora incluye una línea explícita: ✓ **Llaves generadas** o **Llaves sin generar**. Es la misma regla de accesibilidad que ya se aplica al ganador del bracket: ningún estado se comunica solo por color ni solo por la presencia o ausencia de un elemento.

D5 — Sin migración

La historia no toca el esquema. `TotalCompetidores` es un valor **calculado** en la consulta, no una columna, y el usuario Profesor es una **fila** que inserta el seeder. Agregar una columna con el conteo habría sido un dato derivado y desnormalizado que hay que mantener sincronizado con cada alta, baja y clasificación de competidor: una fuente de inconsistencia a cambio de nada.

3. Qué se implementó

Backend

Archivo	Cambio
<code>Infrastructure/Persistence/DbSeeder.cs</code>	Reescrito: lista de semillas, verificación por email, usuario Profesor
<code>Domain/Interfaces/IUsuarioRepository.cs</code>	Se eliminó <code>ExisteAlgunoAsync</code> (quedó sin uso)
<code>Infrastructure/.../UsuarioRepository.cs</code>	Se eliminó la implementación correspondiente
<code>Domain/Interfaces/ICompetidorRepository.cs</code>	Nuevo <code>GetConteoPorCategoriaAsync</code>
<code>Infrastructure/.../CompetidorRepository.cs</code>	Implementación con <code>GroupBy</code> traducido a SQL
<code>Application/Features/Categorias/CategoriaResponse.cs</code>	Nuevo campo <code>int? TotalCompetidores</code>
<code>.../Queries/GetCategoriasByTorneo/...Handler.cs</code>	Injecta <code>ICompetidorRepository</code> y completa el conteo con <code>with</code>

Frontend

Archivo	Cambio
<code>src/types/index.ts</code>	<code>totalCompetidores?: number</code> en <code>Categoria</code>
<code>components/categorias/CategoriaCard.tsx</code>	Fila Competidores + línea de estado de llaves
<code>routes/torneos/\$torneoId/categorias.tsx</code>	Introducción para roles de solo lectura

No hizo falta tocar `lib/api.ts` ni `hooks/useCategorias.ts`: el endpoint y el hook ya existían, y el campo nuevo viaja por el mismo contrato.

4. Tests

`GetCategoriasByTorneoQueryHandlerTests` pasó de 2 a 4 casos:

- **Camino feliz** — mapeo correcto de la categoría (ya existía, adaptado al constructor nuevo).
- **Torneo inexistente** → `NotFoundException` (ya existía).
- **Conteo por categoría** — una categoría con 7 competidores informa 7; una *sin* competidores (ausente del diccionario) informa **0, no null**.
- **Sin N+1** — con 3 categorías, el repositorio de conteo se invoca una sola vez.

Suite completa: 97/97 tests en verde. Backend sin errores C#; frontend compila y pasa lint.

5. Cómo verificarlo

1. Reiniciar la API. El seeder crea al Profesor **sin borrar la base** (D1).
2. Entrar con `profesor@torneo.test` / `Profesor123!`.

3. En la lista de torneos, el Profesor ve *Ver categorías* pero **no** Competidores, ni Editar, ni Eliminar.
4. Entrar a las categorías: se ve la introducción de consulta, y cada card muestra sus rangos, cuántos competidores tiene y si sus llaves están generadas.
5. No aparecen ni el bloque Armar las llaves ni el formulario de nueva categoría.
6. Si hay brackets, el Profesor puede verlos; al abrirlos **no** puede marcar ganadores (`BracketPanel` solo habilita eso al Coordinador).

6. Notas para las próximas historias

- **H0007 (Ver competidores — Profesor)** es el paso siguiente natural. Atención: el endpoint `GET .../competidores/escuela/{escuela}` **no existe**; solo está `GetByEscuelaAsync` en el repositorio. Hay que crear query, handler, endpoint y vista.
- **H0008 (Reporte PDF — Director)** necesitará un usuario Director. Con el seeder de D1 ya idempotente, alcanza con sumar una línea al arreglo de semillas.
- **H0011** convertirá el conteo en dos: un competidor podrá estar a la vez en su categoría de Combate y en la de Formas, así que `GetConteoPorCategoriaAsync` pasará a agrupar sobre la tabla intermedia `competidor_categorias`. La firma del método no cambia.